MODULE 20 · DOMAIN 8 - TOOLS AND MCPS

Tool Use, End to End

Tool definitions, the tool loop, dispatch and errors, parallel calls, client and server tools, tool_choice on current models, approvals for risky tools, and large tool sets.

DOMAIN 8 of 8 EXAM WEIGHT 10.6% MODULE 20 of 36

admin@sthithapragna.com

Claude fills in the form. Your code decides whether to act on it, and does the work.

admin@sthithapragna.com

The tool loop

01
Define
Send tools: name, description, input schema.

02
Request
Claude replies with tool_use blocks; stop_reason is tool_use.

03
Execute
Your code validates the input and runs the tool.

04
Return
Send tool_result blocks with matching tool_use_id.

05
Repeat
Until stop_reason is end_turn, or a limit is hit.

ON THE EXAM

Claude never runs a client tool. Your code runs it, and your code decides whether it runs at all.

A tool definition

```
1  {
2    "name": "tenancy_get",
3    "description": "Look up one tenancy by its ID and return rent, dates, deposit status and "
4                   "the tenants named on it. Use this when a question needs facts about a "
5                   "specific tenancy. Do not use it to search; use tenancy_search for that. "
6                   "Returns an error if the ID does not exist or the caller cannot see it.",
7    "input_schema": {"type": "object",
8                     "properties": {"tenancy_id": {"type": "string"}},
9                     "required": ["tenancy_id"],
10   "strict": true
11 }
```

3-6 What it does, when to use it, when not to, and how it fails.

10 strict: inputs always match the schema.

ON THE EXAM

Anthropic: the description is "by far the most important factor in tool performance".

Designing good tools

- **Descriptions of 3 to 4 sentences or more:** what, when, when not, each parameter, caveats.
- **Consolidate:** one tool with an action parameter beats create_pr, review_pr, merge_pr.
- **Namespace by service:** github_list_prs, slack_send_message.
- **input_examples** show usage; each must be valid against the schema.
- **Return high-signal results** with stable identifiers, not opaque internal references.
- **Ask for an explanation, not reasoning,** in a parameter.

> THE TAKEAWAY
>
> Fewer, better-described tools reduce selection mistakes more than any prompt tweak.

The loop, with a registry and limits

```
1  TOOLS = {"tenancy_get": tenancy_get, "payments_lookup": payments_lookup}   # a registry
2  for turn in range(MAX_TURNS):                                              # a hard stop
3      resp = client.messages.create(model=MODEL, tools=SCHEMAS, messages=msgs)
4      msgs.append({"role": "assistant", "content": resp.content})       # unchanged
5      if resp.stop_reason != "tool_use": break
6      results = []
7      for b in [b for b in resp.content if b.type == "tool_use"]:
8          try:    out, err = json.dumps(TOOLS[b.name](**b.input)), False
9          except ToolError as e: out, err = f"{e}. {e.next_step}", True  # instructive
10         results.append({"type": "tool_result", "tool_use_id": b.id,
11                         "content": out, "is_error": err})
12     msgs.append({"role": "user", "content": results})                # all, in one message
```

1-2 Dispatch by registry; a turn limit the loop cannot exceed.

9, 12 Failures as is_error results; every result in one message.

CURRENCY CHECK

Append the assistant content unchanged, thinking blocks included. Edits cause 400 errors.

Rules for tool results

- **Immediately after:** results must directly follow the tool_use message; nothing in between.
- **Results first:** in that user message, tool_result blocks come before any text. Otherwise, 400.
- **Parallel calls:** run them all, and return every result together, each with its own tool_use_id.
- **Errors:** is_error true, with what went wrong and what to try next, not just "failed".
- **Invalid calls:** Claude retries 2 to 3 times; strict: true eliminates them.

> THE TIEBREAKER
>
> An empty string is not an error message. Claude cannot recover from silence.

Client tools or server tools?

CLIENT TOOLS

Your code runs them

- Your functions; memory, bash, text editor, computer and browser toolsets
- Return tool_result blocks yourself
- Your permissions, your data
- Full control over what runs

SERVER TOOLS

Anthropic runs them

- Web search, web fetch, code execution, tool search
- Results arrive in the same response
- No tool_result needed from you
- pause_turn: send the content back to continue

ON THE EXAM

Anthropic: tools "differ primarily by where the code executes".

tool_choice on current models

SETTING	MEANING	ON OPUS 5.5, SONNET 5.5, FABLE 5.1
`auto`	Claude decides; the default with tools	Supported
`none`	No tool calls	Supported
`any`	Must call some tool	400 error
`tool`	Must call one named tool	400 error

THE TIEBREAKER

Need a guaranteed shape? Use auto with strict tools and clear prompt instructions, or structured outputs.

Risky tools need a gate

- **Separate reads from writes;** give each agent only the tools its job needs.
- **Approval in code** for actions that move money, delete data or contact people.
- **Validate arguments** against business rules, not just the schema.
- **Scope credentials** to the user or task; never one all-powerful key.
- **Idempotency keys** on side effects, as in module 10.

> THE TAKEAWAY
>
> A prompt asking Claude to be careful is guidance. A confirmation step in code is a gate.

Large tool sets

- **Every definition costs input tokens,** and selection gets harder as the list grows.
- **Tool search:** mark rarely used tools defer_loading, and Claude finds them on demand.
- **Keep the few most-used tools** loaded all the time.
- **Consolidate and namespace** before you add search.
- **Anthropic's guidance:** selection accuracy degrades past about 30 to 50 tools.

> ON THE EXAM
>
> Dozens of tools is the signal for consolidation and tool search, not for a longer system prompt.

Copperlet: two planted problems, fixed

QUIRK 5

The tool called get_data

- One tool, no description, free-text argument
- Claude called it for everything
- Now: 6 namespaced tools, 3 to 4 sentence descriptions
- Strict schemas; tested on agent evals

QUIRK 10

The loop that would not stop

- Payments lookup failed: empty string returned
- Retried 30 times, no turn limit
- Now: is_error with the cause and next step
- Turn limit, and a retry budget per tool

SETTLED

Both settled. Clear tools and honest errors fix most agent misbehaviour before any prompt change.

> An agent has two tools, search_customers and search_orders, each with a one-line description. It often calls the wrong one, for example searching customers when asked about a delivery.
>
> The schemas are correct.

What should the developer do first?

A Rewrite each description: what, when, when not, and parameters

B Shorten the tool names, so that they are easier to tell apart

C Force the correct tool with tool_choice for every request

D Add more tools, so each question type has its own

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should the developer do first?

A Rewrite each description: what, when, when not, and parameters

The most important factor, fixed first.

B Shorten the tool names, so that they are easier to tell apart

Names are fine; descriptions are thin.

C Force the correct tool with tool_choice for every request

Code cannot know which is correct, and current models reject it.

D Add more tools, so each question type has its own

More choices, more confusion.

THE SIGNAL

"One-line descriptions" and wrong selections: the description is the fix.

> When an inventory service is down, a tool handler returns an empty string as the tool result. Claude then tells customers that the item is out of stock.
>
> The team wants Claude to respond sensibly when tools fail.

What should the handler return?

A Raise the exception, ending the conversation with an error page

B An is_error tool_result naming the outage and a next step

C A cached stock level from last week, to avoid an error

D No tool_result at all, so that Claude moves on

◆ PAUSE AND DECIDE

What should the handler return?

A Raise the exception, ending the conversation with an error page

Abandons a recoverable conversation.

B An is_error tool_result naming the outage and a next step

Claude can explain, retry or route.

C A cached stock level from last week, to avoid an error

Stale data presented as current.

D No tool_result at all, so that Claude moves on

Breaks the protocol: a 400 error.

THE SIGNAL

Empty results look like real answers. Failures must say they are failures.

> After running a tool, an application sends a user message whose content starts with a text block, "Here are the results:", followed by the tool_result block. Every such request fails with a 400 error.
>
> The tool itself works correctly.

What is the fix?

A Retry the request until it succeeds

B Drop the tool_result, and paste the output into the text block

C Send the tool_result in a later user message

D Put the tool_result block first, and any text after it

◆ PAUSE AND DECIDE

What is the fix?

A Retry the request until it succeeds

A malformed request fails every time.

B Drop the tool_result, and paste the output into the text block

The tool_use still needs its result.

C Send the tool_result in a later user message

Results must follow immediately.

D Put the tool_result block first, and any text after it

The documented order.

THE SIGNAL

Tool results first, text after, in the message straight after the tool_use.

> Claude returns three tool_use blocks in one response, to check three properties at once. The application runs the first tool, sends its result, waits for Claude's next response, then runs the second.
>
> Requests fail, and the flow is slow.

What should the application do?

A Run all three, and return every tool_result in one message

B Run only the first tool, and ignore the other two

C Ask Claude in the prompt never to request more than one tool

D Combine the three results into one tool_result for the first ID

◆ PAUSE AND DECIDE

What should the application do?

A Run all three, and return every tool_result in one message

Every tool_use answered, at once.

B Run only the first tool, and ignore the other two

Two calls left unanswered.

C Ask Claude in the prompt never to request more than one tool

Throws away parallelism.

D Combine the three results into one tool_result for the first ID

Two IDs still unanswered.

THE SIGNAL

Parallel tool_use blocks get parallel execution and one message of results, ID by ID.

> An extraction service forced its single tool with tool_choice set to that tool's name. After moving to Opus 5.5, every request returns a 400 error.
>
> The output must always follow the extraction schema.

What is the best change?

A Switch tool_choice to any, which still guarantees a tool call

B Use auto with a strict tool and a clear instruction, or structured outputs

C Prefill the assistant turn with the start of a tool call

D Return to the previous model permanently

◆ PAUSE AND DECIDE

What is the best change?

A Switch tool_choice to any, which still guarantees a tool call

Also a 400 on this model.

B Use auto with a strict tool and a clear instruction, or structured outputs

The documented replacement.

C Prefill the assistant turn with the start of a tool call

Prefill also returns 400.

D Return to the previous model permanently

Postpones the change, and retirement.

CURRENCY CHECK

Forced tool_choice and prefill are both gone on current models. Auto plus strict, or structured outputs.

> An internal agent has 120 tools, many overlapping, such as separate create, update and close tools for each of a dozen systems. Requests are slow and costly, and Claude often picks the wrong tool.
>
> All the capabilities are still needed.

Which two changes help most? Choose two.

A Use tool search with deferred loading, keeping the most-used tools always loaded

B Copy all the tool descriptions into the system prompt as well

C Consolidate related operations into tools with an action parameter, namespaced by system

D Shorten every description to a few words, to save tokens

E Raise max_tokens, so Claude has room to consider every tool

◆ PAUSE AND DECIDE

Which two changes help most? Choose two.

A Use tool search with deferred loading, keeping the most-used tools always loaded

Fewer definitions in each request.

B Copy all the tool descriptions into the system prompt as well

Doubles the tokens, same confusion.

C Consolidate related operations into tools with an action parameter, namespaced by system

Fewer, clearer choices.

D Shorten every description to a few words, to save tokens

Weakens the most important field.

E Raise max_tokens, so Claude has room to consider every tool

Output limit; unrelated.

THE SIGNAL

Too many overlapping tools: consolidate and namespace, then load the rest on demand.

> A support agent can call issue_refund. In testing, a customer persuaded it to refund an order twice. Refunds above a set amount need a manager's approval under company policy.
>
> The team wants refunds to be safe.

What should the developer do?

A Add a system prompt instruction to be very careful with refunds

B Remove the refund tool, and ask customers to phone instead

C Log all refunds, and review them at the end of each month

D Enforce idempotency, policy checks and approval in code

◆ PAUSE AND DECIDE

What should the developer do?

A Add a system prompt instruction to be very careful with refunds

Guidance; the customer already beat it.

B Remove the refund tool, and ask customers to phone instead

Loses the feature to avoid a gate.

C Log all refunds, and review them at the end of each month

Finds the loss a month later.

D Enforce idempotency, policy checks and approval in code

Gates in code, matching the policy.

THE SIGNAL

A risky side effect gets gates in code: idempotency, rules and human approval.

> An application uses the web search server tool. On long research questions, a response sometimes arrives with stop_reason pause_turn, and the app shows it to the user as the final answer.
>
> Those answers are incomplete.

What should the application do?

A Raise max_tokens, so the response is never paused

B Discard the response, and retry the original request from scratch

C Send the paused response back unchanged, so Claude continues

D Treat pause_turn as an error, and switch to a client-side search tool

◆ PAUSE AND DECIDE

What should the application do?

A Raise max_tokens, so the response is never paused

pause_turn is an iteration limit, not a length one.

B Discard the response, and retry the original request from scratch

Throws away completed work.

C Send the paused response back unchanged, so Claude continues

The documented way to resume.

D Treat pause_turn as an error, and switch to a client-side search tool

Rebuilds a working feature.

THE SIGNAL

pause_turn means a server-tool loop paused, not finished. Send the content back to continue.

Module 20 in five lines

- **The loop:** tool_use, your code validates and runs, tool_result back, until end_turn or a limit.
- **Descriptions matter most;** consolidate, namespace, strict schemas.
- **Results first, immediately, all together;** failures as is_error with a next step.
- **Current models:** no forced tool_choice; auto plus strict. Server tools: handle pause_turn.
- **Risky tools get gates in code;** big tool sets get tool search.

> COMING UP
>
> Next: building MCP servers.